# Disease Prediction Backend — FastAPI & Flask Implementation + Deployment

This document provides ready-to-use backend code (FastAPI and Flask), API formats for the frontend, top-3 prediction & severe-alert endpoints, and CI/CD deployment instructions (Railway, Render, EC2). Copy files into your project and follow the steps.

---

## Contents



---

## 1) Project layout (recommended)

```
backend/
├── app_fastapi.py        # FastAPI server (recommended)
├── app_flask.py         # Flask server (alternative)
├── models/
│   ├── symptom_xgboost_model.pkl
│   ├── disease_encoder.pkl
│   └── symptom_mlb.pkl
├── requirements.txt
├── Dockerfile
├── Procfile             # for Render / Heroku-like platforms
└── README.md
```

---

## 2) Requirements

Use Python 3.9+ (3.10/3.11 recommended). Example `requirements.txt` includes minimal libs:

```
fastapi
uvicorn[standard]
flask
scikit-learn
xgboost
joblib
numpy
pandas
python-multipart
```

Add other libs as needed (gunicorn, python-dotenv, etc.).

---

## 3) FastAPI implementation (recommended)

Create `app_fastapi.py`:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib
import numpy as np
from typing import List, Dict

# Load models once at startup
model = joblib.load("models/symptom_xgboost_model.pkl")
le = joblib.load("models/disease_encoder.pkl")
mlb = joblib.load("models/symptom_mlb.pkl")

# Predefined severe list (tweak as required)
SEVERE_DISEASES = {
    'AIDS', 'Cervical cancer', 'Diabetes', 'Fungal infection',
'Osteoarthristis', 'Varicose veins'
}

app = FastAPI(title="Disease Prediction API")

class SymptomsIn(BaseModel):
    symptoms: List[str]

class VectorIn(BaseModel):
    vector: List[int]

@app.get("/")
def root():
    return {"status": "ok", "message": "Disease Prediction API running"}

@app.post("/predict/by_symptoms")
def predict_by_symptoms(payload: SymptomsIn):
    user_symptoms = payload.symptoms
```

```
    try:
        vector = mlb.transform([user_symptoms])[0]
    except Exception as e:
        raise HTTPException(status_code=400, detail=f"Invalid symptoms. {e}")
    return _predict_from_vector(vector.tolist())

@app.post("/predict/by_vector")
def predict_by_vector(payload: VectorIn):
    vector = payload.vector
    if len(vector) != mlb.transform([[]])[0].shape[0]:
        # fallback: compare length vs mlb + raise helpful error
        expected_len = mlb.transform([[]])[0].shape[0]
        raise HTTPException(status_code=400, detail=f"Vector length must be
{expected_len}")
    return _predict_from_vector(vector)

@app.get("/symptoms")
def list_symptoms():
    # Return the ordered symptom list so frontend can build checkboxes in
same order
    return {"symptoms": list(mlb.classes_)}

@app.get("/health")
def health():
    return {"status": "healthy"}

# Internal util

def _predict_from_vector(vector: List[int]) -> Dict:
    x = np.array(vector).reshape(1, -1)
    probs = model.predict_proba(x)[0]
    top_idx = np.argsort(probs)[-3:][::-1]
    top3 = [{"disease": le.classes_[i], "confidence": float(probs[i])} for i
in top_idx]
    top_pred = top3[0]
    alert = "Consult a professional for confirmation."
    if top_pred["confidence"] > 0.60 and top_pred["disease"] in
SEVERE_DISEASES:
        alert = "See a doctor immediately!"
    return {"prediction": top_pred["disease"], "top3": top3, "alert": alert}

# To run: uvicorn app_fastapi:app --host 0.0.0.0 --port 8000
```

**Notes:** - `/predict/by_symptoms` accepts symptom names (preferable) and uses `mlb.transform`. - `/predict/by_vector` accepts a 0/1 vector if frontend already encodes. - `/symptoms` returns the exact symptom ordering used during training.

---

## 4) Flask implementation (alternative)

Create `app_flask.py`:

```
from flask import Flask, request, jsonify
import joblib
import numpy as np

app = Flask(__name__)

model = joblib.load("models/symptom_xgboost_model.pkl")
le = joblib.load("models/disease_encoder.pkl")
mlb = joblib.load("models/symptom_mlb.pkl")

SEVERE_DISEASES = {
    'AIDS', 'Cervical cancer', 'Diabetes', 'Fungal infection',
'Osteoarthristis', 'Varicose veins'
}

@app.route('/', methods=['GET'])
def root():
    return jsonify({"status": "ok", "message": "Disease Prediction API
running"})

@app.route('/predict/by_symptoms', methods=['POST'])
def predict_by_symptoms():
    data = request.get_json()
    if not data or 'symptoms' not in data:
        return jsonify({"error": "Missing 'symptoms' list"}), 400
    user_symptoms = data['symptoms']
    try:
        vector = mlb.transform([user_symptoms])[0]
    except Exception as e:
        return jsonify({"error": f"Invalid symptoms: {e}"}), 400
    return _predict_from_vector(vector.tolist())

@app.route('/predict/by_vector', methods=['POST'])
def predict_by_vector():
    data = request.get_json()
    if not data or 'vector' not in data:
        return jsonify({"error": "Missing 'vector'"}), 400
    vector = data['vector']
    expected_len = mlb.transform([[]])[0].shape[0]
    if len(vector) != expected_len:
        return jsonify({"error": f"Vector length must be {expected_len}"}),
400
    return _predict_from_vector(vector)

@app.route('/symptoms', methods=['GET'])
def list_symptoms():
```

```
    return jsonify({"symptoms": list(mlb.classes_)})


def _predict_from_vector(vector):
    x = np.array(vector).reshape(1, -1)
    probs = model.predict_proba(x)[0]
    top_idx = np.argsort(probs)[-3:][::-1]
    top3 = [{"disease": le.classes_[i], "confidence": float(probs[i])} for i
in top_idx]
    top_pred = top3[0]
    alert = "Consult a professional for confirmation."
    if top_pred["confidence"] > 0.60 and top_pred["disease"] in
SEVERE_DISEASES:
        alert = "See a doctor immediately!"
    return jsonify({"prediction": top_pred["disease"], "top3": top3, "alert":
alert})

# To run locally for dev:
# export FLASK_APP=app_flask.py
# flask run --host=0.0.0.0 --port=8000
# For production use Gunicorn
```

**Notes:** - Implementation mirrors FastAPI behavior for easy swap. - For production, use `gunicorn -w 4 -k uvicorn.workers.UvicornWorker app_fastapi:app` or `gunicorn app_flask:app`.

---

## 5) Frontend API format

#### 1) Send symptom names (recommended)

`POST /predict/by_symptoms`

Request JSON:

```
{
  "symptoms": ["fever", "fatigue", "headache"]
}
```

Response JSON:

```
{
  "prediction": "Gastroenteritis",
  "top3": [
    {"disease": "Gastroenteritis", "confidence": 0.94},
    {"disease": "Food poisoning", "confidence": 0.03},
    {"disease": "Peptic ulcer", "confidence": 0.02}
  ],
```

```
  "alert": "Consult a professional for confirmation."
}
```

**2) Or send vector if frontend encodes**

`POST /predict/by_vector`

Request JSON:

```
{
  "vector": [0,1,0,0,1, ...]  // length must equal number of symptoms
}
```

**3) Get list of symptoms**

`GET /symptoms` → returns the ordered list `mlb.classes_` so frontend can present checkboxes in correct order.

---

## 6) Top-3 & Severe-alert logic (details)

- `model.predict_proba(x)` returns probability distribution across classes.
- We sort probabilities and pick top 3 indices.
- Alert logic uses 0.60 threshold for high confidence; you can tune.
- `SEVERE_DISEASES` is a set of disease names to trigger immediate alert when predicted with high confidence.

Tune these values or expose them via environment variables as needed.

---

## 7) Example files

### requirements.txt

```
fastapi
uvicorn[standard]
flask
gunicorn
scikit-learn
xgboost
joblib
numpy
pandas
python-multipart
```

### Procfile (for Render/Heroku style)

```
web: uvicorn app_fastapi:app --host 0.0.0.0 --port $PORT
```

---

## 8) Dockerfile (recommended for consistent deploy)

```
FROM python:3.10-slim
WORKDIR /app
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 8000
CMD ["uvicorn", "app_fastapi:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build & run locally:

```
docker build -t disease-api:latest .
docker run -p 8000:8000 disease-api:latest
```

---

## 9) CI/CD & Deployment

### Railway

1. Create a new Railway project → Deploy from GitHub repository.
2. Add environment variables if needed (e.g., `PORT`).
3. Railway will detect `Dockerfile` or `Procfile` and auto-deploy.
4. Ensure `models/` directory (PKL files) is in repo or accessible via build step. For large models, use Railway plugin for file storage or fetch at build time from a private S3.

**Notes:** - Railway's ephemeral filesystem means any runtime writes are temporary. Keep PKLs in repo or use external storage.

### Render

1. Create a new Web Service, connect GitHub repo.
2. Use `Dockerfile` or select a Python environment and set the start command to the same as Procfile.
3. Add any environment variables.
4. Deploy — Render shows logs and provides a URL.

**Notes:** Render supports web services with persistent deployment and custom build commands.

### EC2 (Ubuntu) — simple steps

1. Provision EC2 instance (t3.small or bigger), open port 80/443 and 8000 for testing.

2. SSH into instance.
3. Install Docker (recommended) or Python env.

**Using Docker:**

```
# on server
git clone <your-repo>
cd repo
docker build -t disease-api:latest .
docker run -d --restart always -p 80:8000 disease-api:latest
```

**Without Docker:**

```
sudo apt update && sudo apt install python3-pip -y
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
# run with gunicorn + uvicorn workers
gunicorn -w 4 -k uvicorn.workers.UvicornWorker app_fastapi:app -b 0.
0.0.0:8000
```

Then configure an nginx reverse proxy and SSL (Let's Encrypt) for production.

---

## 10) Notes & Best Practices

- **Load models once** at startup, not per-request.
- **Keep PKL files read-only** and in a secure path. If too large for GitHub, store them in S3 and download at build time.
- **Add logging** for inputs & predictions (PII caution!).
- **Rate limit** or add authentication to prevent abuse.
- **Monitor model drift**: retrain when data distribution changes.
- **Unit tests**: add tests that load the PKLs and run a sample prediction.

---

If you want, I can: - Generate a full repo skeleton (files + Dockerfile + basic tests) - Produce a Postman collection for the endpoints - Create a GitHub Actions workflow for CI/CD

Tell me which you want next and I will generate the files.